



Análise e Desenvolvimento de Sistemas **PROGRAMAÇÃO**
ORIENTADA A OBJETOS COM C# RELATÓRIO DE AULAS
PRÁTICAS

Nome: Vitória Freitas

RA: 2523279

Polo de matrícula: Carapicuíba - Vila Dirce

Local da realização da Aula Prática: Alphaville - Santana de Parnaíba

Ano da postagem: 2026

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): CONSTRUINDO UM PROGRAMA DE CONSOLE EM C# 14, COM .NET 10: TIPOS, NULLABILIDADE, PATTERN MATCHING, RECORDS E PROPRIEDADES

Programação Orientada a Objetos (POO)

A Programação Orientada a Objetos é um paradigma de programação que trabalha com sistemas de entidades e objetos. Os 4 pilares da POO são:

- Abstração: retirar entidades do mundo real e abstrair estes itens nos códigos.
- Encapsulamento: garantir que a alteração de alguns atributos dos objetos são exclusivas de métodos e permitir apenas a visualização.
- Herança: uma entidade pode herdar atributos e métodos de outra entidade.
- Polimorfismo: permite que um mesmo método tenha formas diferentes entre as classes.

As classe, objeto, propriedade, método e encapsulamento podem parecer de diversas formas em programas simples, um exemplo, seria um programa que guarda registros de carros de diversas marcas, teria uma classe chamada 'Carro', contendo propriedades (como: 'placa', 'marca', 'kilometros_rodados' e 'preco') e metodos (como: 'alterar_preco', 'acessar_preco' e 'vender'). Cada carro dentro do sistema seria uma instância da classe 'Carro', assim criando diversos objetos, o encapsulamento seria alcançado ao privar de alterar o preço e apenas permitir a visualização dele, para alterar seria através do método, com isso teríamos um programa simples que possui classe, objeto, propriedade, método e encapsulamento.

Tipos por valor e tipos por referência

Tipos por valor são aqueles que armazenam diretamente os seus dados, eles contém em si o valor atribuído, os tipos: int, double, bool, char e struct são tipos por valor. Os tipos por referência armazenam endereços para os objetos alocados no heap, podemos citar como exemplo deles: Class, array e string.

Nullabilidade

A nulabilidade está presente em C# presentes nos tipos anuláveis por valor, eles usam uma sintaxe de interrogação, por exemplo, int?. Assim um tipo por valor pode conter um valor null.

Pattern matching

Pattern matching é uma técnica de testar uma expressão para verificar se ele possui determinadas características, provendo meios com uma sintaxe mais consistente para realização de testes de expressão e quando a expressão passar

nesse teste o programa realizará uma ação. Exemplo:

```
int? valor = 10;
if (valor is not null)
{
    Console.WriteLine($"O valor é igual a {valor}");
};
```

Records e with-expression

O record prover funcionalidade de encapsulamento dos dados de maneira embutida. Quando se declara um construtor primário no record o compilador irá gerar propriedades públicas para os parâmetros dele. Quando define uma classe como record isso significa que os objetos desse tipo serão tratados com valores imutáveis, exemplo:

```
public record Carro
{
    public string Marca { get; init; }
    public string Placa { get; init; }
};
```

O record fornece suporte para a with-expression, com ela podemos criar cópias do objeto com propriedades diferentes, exemplo:

```
var carro1 = new Carro
{
    Marca = "FIAT",
    Placa = "5435-3255"
};
var carro2 = carro1 with { Placa = "4324 - 4317" };
```

Propriedade com validação

Com as propriedades com validação podemos verificar um valor antes de atribuir um valor a ele, caso recebamos um valor null podemos retornar um ArgumentException. E também podemos encapsular os valores em campos privados para garantir uma integridade ao dados como o exemplo a seguir:

```
private int _saldo;
public int Saldo
{
    get => _saldo;
```

```

        set {
            if (value < 0) throw new ArgumentException("Valor invalido");
            _saldo = value;
        }
    }
}

```

Atribuição condicional nula

A atribuição condicional nula permite que uma atribuição ocorra condicionalmente, como nesse exemplo:

```

void Exemplo(Valor? v)
{
    v?.Numero = 20; // caso não seja nulo, o valor será atribuído
}

```

Código-fonte comentado

```

public record Produto
{
    public int Id { get; init; }
    public string Nome { get; init; }
    public double Preco { get; init; }
};

public class Config
{
    private string _tema;
    public string Tema
    {
        get => _tema;

        set
        {
            if (value == null) throw new ArgumentNullException("Valor nulo é inválido.");
            _tema = value;
        }
    }
}

```

```

}
public class Cliente
{
    public decimal Total { get; set; }
}
namespace relatorio1
{
    class Program
    {
        static void Main()
        {
            //Etapa - 1: Imprimindo uma saldação na tela
            Console.WriteLine("Olá, bom dia");
            //Etapa - 2: Declarando variáveis tipos por valor
            int idade = 19;
            string nome = "Isaac";
            Console.WriteLine($"Olá, meu nome é {nome} e tenho {idade} anos.");
            //Etapa 3 - Nullabilidade
            string? frase = null;
            string exibir = frase ?? "Valor não atribuído";
            Console.WriteLine(exibir);
            frase ??= "O dia está ótimo";
            Console.WriteLine(frase);
            //Etapa 4 - switch como expressão
            object? valor = 120;
            string descricao = valor switch
            {
                null => "Valor nulo",
                string => "Valor é uma string",
                int x when x > 100 => "Valor inteiro maior que 100",
                _ => "Valor não encontrado"
            };
            Console.WriteLine(descricao);
        }
    }
}

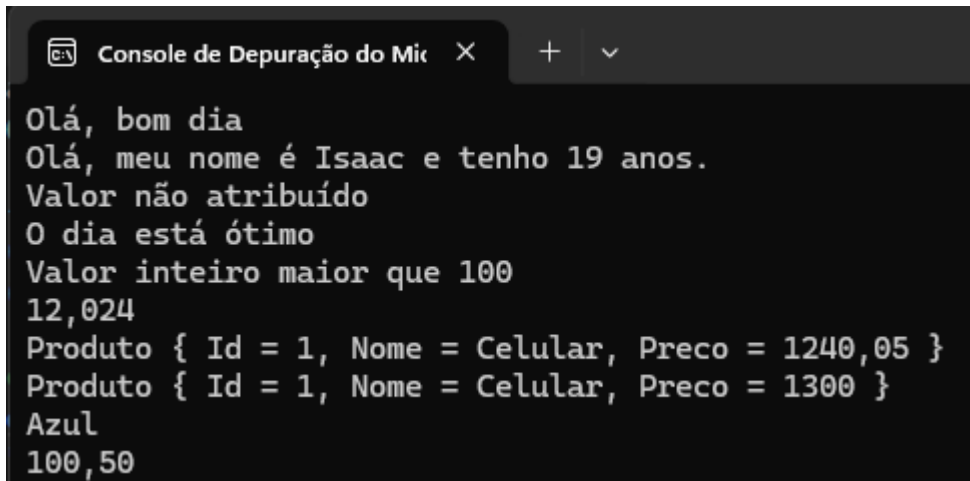
```

```

//Etapa 5 - Função local static
static decimal Calc(decimal preco, decimal valor = 0.12m) => preco * valor;
decimal resultado = Calc(100.2m);
Console.WriteLine(resultado);
//Etapa 6 - record
var p1 = new Produto
{
    Id = 1,
    Nome = "Celular",
    Preço = 1240.05
};
var p2 = p1 with { Preço = 1300.00 };
Console.WriteLine(p1);
Console.WriteLine(p2);
//Etapa 7 - Classe com propriedade e o token contextual field
var newConfig = new Config();
newConfig.Tema = "Azul";
Console.WriteLine(newConfig.Tema);
//Etapa 8 - Classe Cliente e atribuição condicional com ?. no destino
Cliente? cliente = new Cliente();
cliente?.Total += 100.50m;
Console.WriteLine(cliente?.Total);
}
}
}

```

Resultado no terminal:



```
Olá, bom dia
Olá, meu nome é Isaac e tenho 19 anos.
Valor não atribuído
O dia está ótimo
Valor inteiro maior que 100
12,024
Produto { Id = 1, Nome = Celular, Preço = 1240,05 }
Produto { Id = 1, Nome = Celular, Preço = 1300 }
Azul
100,50
```

Referências

DIO. **POO: programação orientada a objetos – guia completo**. Disponível em: <https://www.dio.me/articles/poo-programacao-orientada-a-objetos-guia-completo>. Acesso em: 24 abr. 2026.

UNIVERSIDADE PAULISTA (UNIP). **Livro-texto – Unidade I: Programação Orientada a Objetos com C#**. Disponível em: https://ava.ead.unip.br/bbcswebdav/pid-5065360-dt-content-rid-17960334_1/institution/Conte%C3%BAdos_AVA/DISCIPLINAS_GERAIS/8120-60%20-%20Programa%C3%A7%C3%A3o%20Orientada%20a%20Objetos%20com%20C%23/LibroTexto%20-%20Unidade%20I.pdf. Acesso em: 24 abr. 2026.

MICROSOFT. **Pattern matching in C#**. Disponível em: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/functional/pattern-matching>. Acesso em: 24 abr. 2026.

MICROSOFT. **Record types in C#**. Disponível em: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/record>. Acesso em: 24 abr. 2026.

MACORATTI, José Carlos. **C# 9.0 – usando records (parte 1)**. Disponível em: https://www.macoratti.net/20/11/c9_record1.htm. Acesso em: 24 abr. 2026.

MICROSOFT. **Usando propriedades (C# Programming Guide)**. Disponível em: <https://learn.microsoft.com/pt-br/dotnet/csharp/programming-guide/classes-and-structs/using-properties>. Acesso em: 24 abr. 2026.

MICROSOFT. **Null-conditional assignment (C# 14.0 proposal)**. Disponível em: <https://learn.microsoft.com/pt-br/dotnet/csharp/language-reference/proposals/csharp-14.0/null-conditional-assignment>. Acesso em: 24 abr. 2026.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): DESENVOLVENDO UM PROGRAMA DE CONSOLE EM C#: AS CONDICIONAIS E OS LAÇOS (COM .NET 10 E C# 14)

Instruções condicionais

Estruturas de controles, como if e else permitem que o programa faça ações baseadas em condições, caso seja verdadeira ele vai tomar uma ação, caso seja falsa tomará outra diferente, assim controlando o fluxo do programa.

O que é o switch

O switch é uma estrutura condicional para selecionar um caso entre várias opções de blocos de códigos que podem ser executadas, dependendo do resultado da expressão ele irá executar um deles. Estrutura em c#:

```
switch (expression)
{
    case x:
        // código do bloco
        break;
    case y:
        // código do bloco
        break;
    default:
        // código do bloco
        break;
}
```

Pode ser utilizado no lugar de if e else encadeados, trazendo uma melhor legibilidade e organização para código.

O que são os laços de repetição (for e while)

Laços de repetição são estruturas de condição que vão executar repetidamente um bloco de código até uma determinada expressão for verdadeira. O while é ideal quando não sabemos quantas vezes o laço irá repetir, o for é ideal

quando o laço será executado uma determinada quantidade de vezes.

Case-sensitive

Case-sensitive é quando um programa é sensível a letras minúsculas e maiúsculas, portanto uma variável X é uma variável x serão consideradas diferentes para o programa.

O papel de ponto e vírgula (;) e de chaves ({ e }) na estrutura do código.

O papel do ponto e vírgula ele diz quando uma linha de código acabou e as chaves para agrupar diversos comandos em bloco de código único.

Como métodos e classes podem organizar um programa que inicialmente foi escrito no arquivo principal.

Com os métodos e classes podemos organizar as funções do programa de uma melhor maneira, assim cada método cumprirá uma funcionalidade diferente.

O código-fonte comentado

Program.cs

```
namespace relatorio2;
class Program
{
    static void Main(string[] args)
    {
        SystemClass system = new SystemClass();
        //Etapa 1 — Mensagens iniciais no console
        Console.WriteLine("Demonstração de condicionais e laços");
        Console.WriteLine();
        //Etapa 2 — Identificadores case -sensitive
        system.CaseSensitiveCase();
        //Etapa 3 — Classificação por nota com if / else if / else | Testamos com esses números: 40, 70 e 85
        system.GradeClassifier(85);
        //Etapa 4 — Comando com switch | Testamos com esses comandos: A, B, C e D
        system.SwitchCommand('A');
        //Etapa 5 — Contagem com o laço for | Testamos com esses números: 5, 10, 15
        system.Count(5);
        //Etapa 6 — Contagem regressiva com o laço while | Testamos com esses números: 3, 5, 10
        system.RegressiveCount(3);
        string? entrada = null;
        do
        {
            Console.WriteLine("\nAperte enter para encerrar o console.");
            entrada = Console.ReadKey().Key.ToString();
        } while (entrada != "Enter");
    }
}
```

SystemClass.cs

```
namespace relatorio2
{
    public class SystemClass
    {
        public void CaseSensitiveCase()
        {
            int aluno = 10;
```

```

int Aluno = 20;
Console.WriteLine($"aluno: {aluno}");
Console.WriteLine($"Aluno: {Aluno}");
}
public void GradeClassifier(int nota)
{
    if (nota < 50)
    {
        Console.WriteLine("Desempenho ruim");
    }
    else if (nota < 70)
    {
        Console.WriteLine("Desempenho mediano");
    }
    else if (nota < 90)
    {
        Console.WriteLine("Desempenho bom");
    }
    else
    {
        Console.WriteLine("Desempenho sensacional");
    }
}
public void SwitchCommand(char comando)
{
    //Expressão switch, guardando resultado em variável
    /*
    string mensagem = comando switch
    {
        'A' => "Comando A",
        'B' => "Comando B",
        'C' => "Comando C",
        _ => "Comando desconhecido"
    };
    Console.WriteLine(mensagem);
    */
    switch (comando)
    {
        case 'A':
            Console.WriteLine("Comando A");
            break;
        case 'B':
            Console.WriteLine("Comando B");
            break;
        case 'C':
            Console.WriteLine("Comando C");
            break;
        default:
            Console.WriteLine("Comando desconhecido");
            break;
    }
}
public void Count(int value)
{
    Console.WriteLine($"Contar até {value}");
    for (var i = 1; i <= value; i += 1)
    {
        Console.WriteLine(i);
    }
}
public void RegressiveCount(int contador)
{
    Console.WriteLine("Contagem regressiva:");
    while (contador >= 1)
    {

```

```
        Console.WriteLine(contador);  
        contador -= 1;  
    }  
    Console.WriteLine("Contagem encerrada.");  
}  
}
```

Resultado no terminal:

```
Console de Depuração do Mic X + v  
Demonstração de condicionais e laços  
aluno: 10  
Aluno: 20  
Desempenho bom  
Comando A  
Contar até 5  
1  
2  
3  
4  
5  
Contagem regressiva:  
3  
2  
1  
Contagem encerrada.  
Aperte enter para encerrar o console.
```

Referências (ABNT)

DIO. **Operadores condicionais em C#: como o if, if aninhado e switch moldam o fluxo do seu programa para respostas inteligentes.** Disponível em: DIO. Acesso em: 27 abr. 2026.

W3SCHOOLS. **C# Switch Statement.** Disponível em: W3Schools. Acesso em: 27 abr. 2026.

DIO. **Laços de repetição.** Disponível em: DIO. Acesso em: 27 abr. 2026.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): IMPLEMENTANDO UM MINIJOGO DE BATALHA EM CONSOLE COM POO EM C# (CLASSES E OBJETOS)

1) ORIENTAÇÕES GERAIS (ENTREGA)

Cada aluno (ou equipe) deverá entregar **um relatório curto (2 a 3 páginas)** contendo:

1.1) Um resumo teórico (com palavras próprias)

Inclua, obrigatoriamente:

- A definição de **classe** e de **objeto (instância)** e a diferença entre os dois.
- A explicação do que são **propriedades, métodos e construtores** em C#.
- A explicação do que é **encapsulamento**, com um exemplo no seu código (por exemplo, uma propriedade com set restrito).
- A explicação do que é uma **propriedade calculada** (por exemplo, um bool que depende de Vida).
- A descrição de como ocorre a **interação entre objetos** no minijogo (um objeto chamando método de outro e alterando o estado do outro).
- A descrição do uso de **condicionais** e de **laços**, com foco no loop de turnos da batalha.

1.2) O código-fonte completo, com comentários didáticos

- Cole o código final no relatório.
- Faça comentários em trechos relevantes, como:
 - a criação de objetos com new;
 - o construtor e as inicializações;
 - o uso de get, set e private set;
 - o método de ataque e o método de receber dano;
 - o laço while da batalha;
 - as verificações de “estar vivo”.

1.3) Um registro de testes

Inclua pelo menos:

- um teste que mostre **uma batalha completa** contra o primeiro inimigo;
- um teste que mostre **a segunda batalha** (se o jogador sobreviver);
- um teste que confirme que a vida **não fica negativa**.

1.4) As referências em ABNT

- Liste os livros (e, se necessário, páginas oficiais) que você consultou.
- A formatação deve seguir a ABNT (modelo no final deste roteiro).

2) REQUISITOS FUNCIONAIS DO PROGRAMA

Ao final, seu programa deve:

1. Exibir um título do minijogo e uma breve explicação.
2. Criar **um objeto** de jogador (por exemplo, “Herói”) com vida e força iniciais.
3. Criar **dois objetos** de inimigos diferentes (por exemplo, “Slime” e “Goblin”), com vida, força e recompensa.
4. Mostrar o estado inicial do jogador e dos inimigos no console.
5. Executar **duas batalhas em sequência**:
 - o primeira batalha: jogador vs. inimigo 1;
 - o segunda batalha: jogador vs. inimigo 2 (somente se o jogador estiver vivo).
6. Em cada batalha:
 - o ocorrerem turnos;
 - o o jogador atacar;
 - o o inimigo contra-atacar, se ainda estiver vivo;
 - o existir uma pausa por ENTER para avançar os turnos.
7. Ao derrotar um inimigo, o jogador ganhar pontos de recompensa.
8. Exibir o estado final do jogador ao término das batalhas.

3) REQUISITOS DE POO (O QUE DEVE EXISTIR EM CÓDIGO)

Você deve implementar as seguintes classes (com nomes iguais ou equivalentes):

3.1) A classe Jogador

Implemente:

- as propriedades:
 - o Nome (texto);
 - o Vida (inteiro) com **leitura pública e alteração restrita** (por exemplo, private set);
 - o Força (inteiro);
 - o Pontuacao (inteiro) com **alteração restrita**;
 - o EstaVivo (booleano) como **propriedade calculada**, baseada em Vida.
- o construtor:
 - o deve receber nome, vidaInicial e forçaInicial;
 - o deve inicializar Pontuacao com zero.
- os métodos:
 - o Atacar(Inimigo alvo):
 - deve impedir ataque se o jogador estiver sem vida;
 - deve impedir ataque se o alvo já estiver derrotado;
 - deve causar dano igual à força;
 - deve chamar alvo.ReceberDano(...).
 - o ReceberDano(int dano):

-
- deve reduzir a vida;
 - deve impedir que a vida fique negativa (mínimo zero);
 - deve escrever o novo valor no console.

- GanharPontos(int pontos):
 - deve somar na pontuação;
 - deve escrever a pontuação no console.

3.2) A classe Inimigo

Implemente:

- as propriedades:
 - Nome;
 - Vida com alteração restrita;
 - Força;
 - PontosDeRecompensa;
 - EstaVivo como propriedade calculada.
- o construtor:
 - deve receber nome, vidaInicial, forçaInicial e pontosDeRecompensa.
- os métodos:
 - Atacar(Jogador alvo) com validações análogas às do jogador;
 - ReceberDano(int dano) com vida mínima zero e mensagens no console.

3.3) A classe Jogo

Implemente:

- o método Executar() que:
 - cria os objetos;
 - imprime os estados iniciais;
 - chama o método de batalha para o primeiro inimigo;
 - encerra se o jogador morrer;
 - chama o método de batalha para o segundo inimigo;
 - imprime o estado final do jogador.
- um método auxiliar Batalhar(Jogador jogador, Inimigo inimigo) que:
 - executa um laço while enquanto ambos estiverem vivos;
 - realiza turnos com ataque e contra-ataque;
 - pausa por ENTER a cada turno;
 - ao final, concede pontos ao jogador se ele vencer.

3.4) A classe Program

Implemente:

- o método Main() que:
 - cria um objeto Jogo;
 - chama Executar().

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	Clareza, correção conceitual e boa explicação das condicionais, dos laços e do case-sensitive.
A estrutura e a organização do código	3,0	Indentação correta, nomes adequados, comentários úteis e organização coerente.
O funcionamento da solução	3,0	Compilação e execução sem erros, com saídas coerentes em todos os testes solicitados.
A criatividade e os aprimoramentos	2,0	Implementação de extensões, validações, melhorias de interação e organização em classes/métodos.

Nota Final: Será a soma ponderada dos itens. Entregas fora do prazo ou com funcionamento mínimo incompleto terão desconto proporcional.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): IMPLEMENTANDO UMA CONTA BANCÁRIA COM POO EM C# (ENCAPSULAMENTO, PROPRIEDADES E MÉTODOS)

1) ORIENTAÇÕES GERAIS (ENTREGA)

Cada aluno (ou equipe) deverá entregar **um relatório curto (2 a 3 páginas)** contendo:

1.1) Um resumo teórico (com palavras próprias)

Inclua, obrigatoriamente:

- A explicação do que é **POO**, com foco em **classe** e **objeto**.
- A explicação do que é **encapsulamento** e por que se evita expor diretamente o estado interno.
- A diferença entre **campo** (por exemplo, saldo) e **propriedade** (por exemplo, Saldo).
- A explicação de um **get público** e de um **set restrito** (por exemplo, private set).
- A justificativa do uso de decimal para valores monetários.
- A descrição do papel de um **construtor** com valor padrão (parâmetro opcional).
- A explicação de como **métodos** podem impor regras de negócio (por exemplo, não aceitar depósito negativo e não permitir saque maior que o saldo).

1.2) O código-fonte completo, com comentários didáticos

- Cole o código final no relatório.
- Comente trechos relevantes:
 - o campo privado;
 - a propriedade de leitura pública;
 - o construtor com saldoInicial = 0m;
 - as validações de depósito e de saque;
 - as alterações no saldo.

1.3) Um registro de testes

Inclua pelo menos:

- um teste de depósito válido;
- um teste de depósito inválido (zero ou negativo);
- um teste de saque válido;
- um teste de saque inválido (maior que o saldo);
- um teste de saque inválido (zero ou negativo).

1.4) As referências em ABNT

- Liste os livros (e, se necessário, páginas oficiais) consultados.
- Formate em ABNT (modelo no final deste roteiro).

2) OBJETIVO DA ATIVIDADE

Construir uma classe ContaBancaria que mantenha um saldo com segurança (encapsulado) e ofereça operações de **depósito** e **saque** com validações, além de uma aplicação Console simples para testar o comportamento.

3) REQUISITOS DO PROGRAMA

3.1) Requisitos de POO (classe principal)

Você deve implementar uma classe chamada ContaBancaria que contenha:

1. **Um campo privado** saldo do tipo decimal.
2. **Uma propriedade pública** Saldo do tipo decimal com:
 - um **get público**;
 - um **set restrito** (por exemplo, private set) ou uma alternativa equivalente que impeça alterações externas.
3. **Um construtor** que receba um saldoInicial do tipo decimal com valor padrão (por exemplo, zero).
4. **Um método** Depositar(decimal quantia) que:
 - rejeite quantias menores ou iguais a zero;
 - some a quantia ao saldo quando válida;
 - retorne true quando efetuar o depósito e false quando rejeitar.
5. **Um método** Sacar(decimal quantia) que:
 - rejeite quantias menores ou iguais a zero;
 - rejeite saques acima do saldo;
 - subtraia a quantia do saldo quando válida;
 - retorne true quando efetuar o saque e false quando rejeitar.

3.2) Requisitos de aplicação (Console)

No Main, crie um pequeno roteiro de execução que:

- crie pelo menos **uma** conta com saldo inicial;
- tente realizar depósitos e saques válidos e inválidos;
- imprima no console:
 - o saldo inicial;
 - o resultado (sucesso/fracasso) de cada operação;
 - o saldo após cada operação.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A clareza e a correção dos conceitos de classe, objeto, encapsulamento, campo, propriedade, construtor e métodos.
A estrutura e a organização do código	3,0	A indentação correta, os nomes adequados, os comentários úteis e a separação clara entre a classe e o programa de testes.
O funcionamento da solução	3,0	A execução sem erros, as validações corretas, o saldo consistente e os testes demonstrando os casos válidos e inválidos.
A criatividade e os aprimoramentos	2,0	A implementação de extensões opcionais com consistência e boa integração ao código existente.

Nota Final: Será a soma ponderada dos itens. Entregas fora do prazo ou com funcionamento mínimo incompleto terão desconto proporcional.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ABSTRAÇÃO, HERANÇA E POLIMORFISMO EM C# (APLICAÇÃO DE CONSOLE)

OBJETIVO

Implementar, em uma aplicação de console, uma hierarquia simples de classes para modelar animais, usando:

- a classe abstrata (para definir um “contrato” comum);
- a herança (para reutilizar e especializar comportamento);
- o polimorfismo (para chamar o mesmo método em objetos de tipos diferentes).

ORIENTAÇÕES

Cada aluno (ou equipe) deve produzir **um relatório curto (1 a 2 páginas)** contendo:

1) O resumo teórico

No texto, explique com palavras próprias:

- o que é **Programação Orientada a Objetos (POO)** e o que se pretende ao usar POO;
- o que é **uma classe abstrata** e por que ela é útil em um projeto;
- a diferença entre **método abstrato** e **método virtual** (se aparecer na sua explicação, compare corretamente);
- o que é **herança** e quando ela faz sentido;
- o que é **polimorfismo** no contexto de “uma variável do tipo base apontando para um objeto do tipo derivado”;
- o papel de **construtores** em uma classe base e em uma classe derivada;
- o que é **override** e por que ele é exigido ao implementar um método abstrato.

2) O código-fonte comentado

- inserir **o código-fonte completo** da solução produzida;
- comentar as linhas mais relevantes, com foco em:
 - a declaração da classe abstrata;
 - a propriedade que guarda o nome;
 - o construtor da classe base e a chamada do base(...) no construtor das subclasses;
 - a sobrescrita (override) do método que emite som;
 - os pontos do programa em que aparece o polimorfismo.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A clareza, a correção dos conceitos e a conexão entre abstração, herança e polimorfismo.
A estrutura e a organização do código	3,0	A identificação correta, os nomes adequados, a separação coerente em classes e os comentários necessários.
O funcionamento da solução	3,0	A compilação sem erros e a execução com saídas coerentes, incluindo chamadas polimórficas.
A criatividade e os aprimoramentos	2,0	A inclusão de extensões (por exemplo, novos animais, listas, entradas do usuário, validações) com qualidade.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não entregarem no prazo ou não atingirem o funcionamento mínimo terão desconto proporcional na nota.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): INTERFACES E POLIMORFISMO EM C# (GEOMETRIA EM APLICAÇÃO DE CONSOLE)

OBJETIVO

Construir uma aplicação de console que modele **formas geométricas** por meio de **uma interface**, com classes concretas que a implementam, e que use **polimorfismo** ao armazenar objetos diferentes em uma coleção do tipo List<IForma>.

ORIENTAÇÕES

Cada aluno (ou equipe) deve produzir **um relatório curto (1 a 2 páginas)** contendo:

1) O resumo teórico

Explique, com palavras próprias:

- o que é **uma interface** em POO e qual é a ideia de “contrato” (quais métodos devem existir);
- a diferença entre **interface** e **classe abstrata** (com exemplos do que cada uma costuma resolver);
- o que é **implementação** de interface em uma classe concreta;
- o que é **polimorfismo** com interface (por exemplo, “uma variável do tipo IForma pode referir-se a objetos de classes diferentes”);
- o papel de **propriedades** e de **construtores** para guardar e inicializar os dados (largura, altura, raio);
- a diferença entre **área** e **perímetro**, com as fórmulas que você usou no código.

2) O código-fonte comentado

- inserir **o código-fonte completo** da solução produzida;
- comentar as partes principais, incluindo:
 - a interface e as assinaturas dos métodos;
 - as propriedades e os construtores das classes;
 - os cálculos em CalcularArea() e em CalcularPerimetro();
 - o uso de List<IForma> e do laço que percorre as formas;
 - a impressão formatada no console.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A correção conceitual sobre interfaces, implementação e polimorfismo, com explicações claras.
A estrutura e a organização do código	3,0	A boa organização em interface e classes, a identificação correta, os nomes adequados e os comentários necessários.
O funcionamento da solução	3,0	A compilação sem erros, a execução correta e os cálculos coerentes de área e de perímetro.
A criatividade e os aprimoramentos	2,0	A inclusão de extensões com validações, novas formas, entradas do usuário ou melhor apresentação da saída.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não entregarem no prazo ou não atingirem o funcionamento mínimo terão desconto proporcional na nota.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): Modelando herança e polimorfismo com as classes Animal e Cachorro em C#

Objetivo geral

Implementar, compilar e executar um pequeno exemplo de Programação Orientada a Objetos em C#, usando **uma classe base** e **uma classe derivada**, para praticar **as propriedades, os métodos, a herança e o polimorfismo** (com virtual e override).

Orientações para a entrega (relatório curto, de 1 a 2 páginas)

Cada aluno (ou equipe) deve entregar um relatório contendo:

1) Um resumo teórico (com palavras próprias)

Inclua, de forma objetiva:

- **A definição de classe e de objeto**, distinguindo “o modelo” (a classe) de “uma instância” (o objeto).
- **A explicação de encapsulamento**, destacando o papel de **propriedades** (por exemplo, Nome) e de **métodos** (por exemplo, Comer()).
- **A explicação de herança**, descrevendo como uma classe derivada reaproveita e especializa uma classe base.
- **A explicação de polimorfismo**, relacionando:
 - **o método virtual na classe base**, e
 - **o método override na classe derivada**, e descrevendo o que muda no comportamento quando o método é sobrescrito.

2) Um código-fonte comentado

- Inserir **o código-fonte completo** que você produziu.
- Comentar **as linhas principais**, incluindo:
 - a declaração da **classe base**,
 - a declaração da **classe derivada**,
 - a propriedade Nome,
 - a chamada de Console.WriteLine(...),
 - o uso de virtual e de override,
 - a criação de objetos (instâncias) e a chamada de métodos no Main.

3) As referências em ABNT

- Listar **os livros e os materiais consultados**.
- Formatar as referências conforme **as regras da ABNT**.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A clareza, a correção conceitual e a relação entre classe, objeto, herança e polimorfismo.
A estrutura e a organização do código	3,0	A organização, os nomes adequados, os comentários úteis e a legibilidade.
O funcionamento da solução	3,0	A execução sem erros, a criação correta das classes e o uso correto de virtual e override.
A criatividade e os aprimoramentos	2,0	A inclusão de extensões, como um novo animal, uma lista polimórfica e validações.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): Herança, polimorfismo e casting com Animal e Cachorro em C#

Orientações

Cada aluno (ou equipe) deve produzir um relatório curto (1 a 2 páginas) contendo:

1) Um resumo teórico (com palavras próprias)

No resumo, inclua:

- **A definição de classe e de objeto**, explicando a relação entre “tipo” e “instância”.
- **A explicação de herança**, descrevendo o que significa Cachorro : Animal e o que é reutilizado.
- **A explicação de polimorfismo**, descrevendo:
 - o que significa marcar um método como virtual na classe base;
 - o que significa marcar um método como override na classe derivada;
 - por que a chamada de FazerSom() pode executar implementações diferentes, dependendo do tipo real do objeto.
- **A explicação de referência estática e tipo dinâmico**, usando um exemplo do tipo:
 - “uma variável do tipo Animal pode referenciar um objeto do tipo Cachorro”.
- **A explicação de casting (conversão de tipo)**, respondendo:
 - por que animal2.AbanarRabo() não compila;
 - por que ((Cachorro)animal2).AbanarRabo() compila;
 - o que acontece se o objeto não for realmente um Cachorro (erro em tempo de execução).
- **A distinção entre erro de compilação e erro em tempo de execução**, aplicando aos trechos do programa.

2) Um código-fonte comentado

- Inserir o código-fonte completo que você produziu.
- Comentar as partes essenciais:
 - as classes Animal e Cachorro;
 - a propriedade Nome;
 - o método Comer() e as mensagens no console;
 - o método virtual FazerSom() e o override em Cachorro;
 - a criação de animal1 e de animal2;
 - a diferença entre Animal animal2 = new Cachorro(); e Animal animal1 = new Animal();;
 - o motivo do comentário que indica erro em animal2.AbanarRabo();
 - o casting ((Cachorro)animal2) e a chamada de AbanarRabo() após o cast.

3) As referências em ABNT

- Listar os livros e os materiais consultados, em formato ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A correção e a clareza nos conceitos de classe, objeto, herança, polimorfismo, casting e tipos.
A estrutura e a organização do código	3,0	A indentação, os nomes claros, os comentários úteis e a organização em blocos.
O funcionamento da solução	3,0	A compilação e a execução sem erros, com saídas corretas e demonstrações pedidas (inclusive o cast).
A criatividade e os aprimoramentos	2,0	A inclusão de melhorias, como verificação de tipo com is, uso de as, novas classes e listas polimórficas.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.